

CS3012-Natural Language Processing

S. Divya

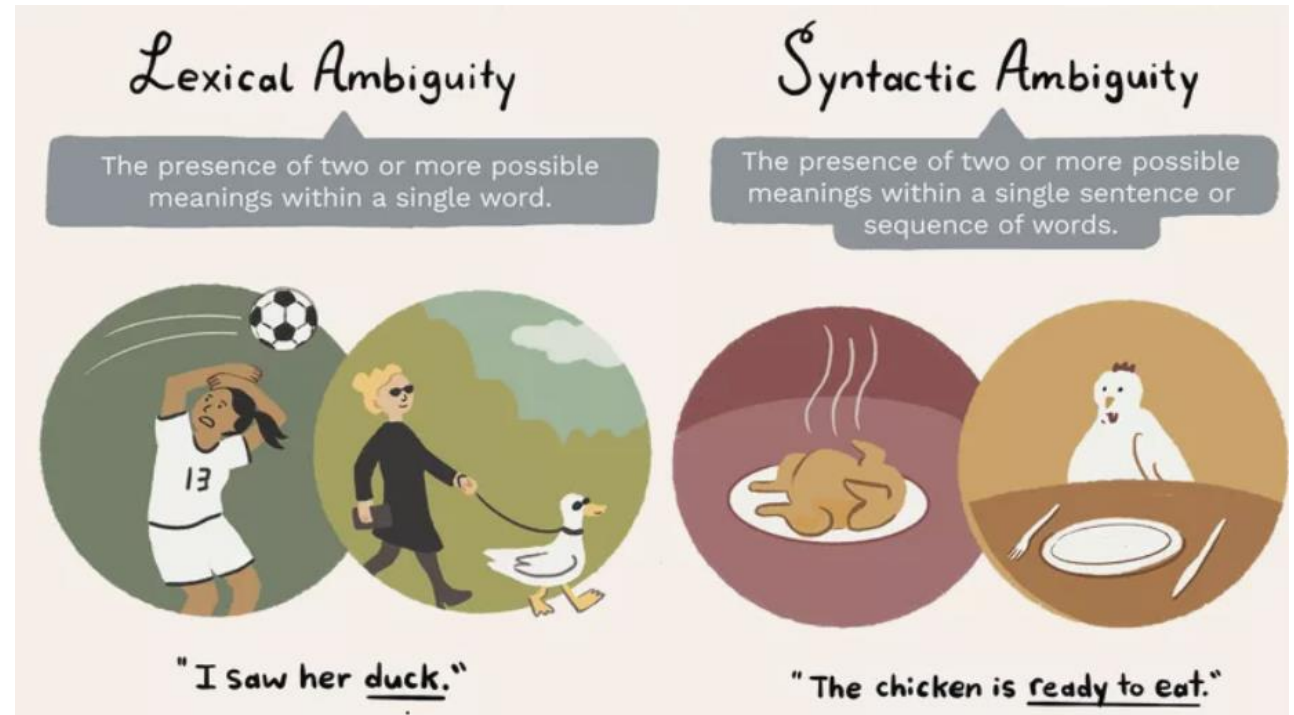
Assistant Professor

Computer Science and Engineering

N-Gram Language Models

Human Word Prediction

- We may have ability to **predict future words** in an utterance
- Ex: Alex swallowed the large green ____.
 - A. Frog
 - B. Mountain
 - C. Car
 - D. Pill
- How?
 - Domain Knowledge
 - The fisherman went to the bank
 - Syntactic knowledge
 - The chicken is ready to eat
 - Lexical knowledge
 - I saw her duck
 - Baked <potato>



Context Sensitive Spelling Correction

- Example: The office is about fifteen **minuets** from my house

min·u·et  *noun* \,min-yə-'wet\
: a slow, graceful dance that was popular in the 17th and 18th centuries
: the music for a minuet

Use a Language Model

$P(\text{about fifteen } \mathbf{minutes} \text{ from}) > P(\text{about fifteen } \mathbf{minuets} \text{ from})$

Speech Recognition

- $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

Machine Translation

Which sentence is more plausible in the target language?

- $P(\mathbf{high} \text{ winds}) > P(\mathbf{large} \text{ winds})$

Introduction -Language models

- Predicting the next few words in a sentence -- Possibility of *assigning a probability score to the next word in the sentence*.
- For example: what word is likely to follow:
How are you
How do you do
How old are you
- why would we want to assign *probability score* to a sentence? --probabilities are essential to identify words in a noisy input like **speech recognition**.
- It can also help in task like **spelling correction** or **grammatical error correction**.
- It has excessive usage in field of **machine translation**.

N-gram language model

- Language Model (LM)
 - A language model is a probability distribution over entire sentences or texts
 - N-gram: unigrams, bigrams, trigrams,...
- In a simple *n-gram language model*, the probability of a word, conditioned on some number of previous words
- An N-gram language model **predicts the probability of a given N-gram within any sequence of words in the language.**
- In other words, using the **previous N-1 words in a sequence we want to predict the next word** (similar to human word prediction)

What is an N-gram?

- An ***n*-gram** is a subsequence of n items from a given [sequence](#).
 - Unigram: n -gram of size 1
 - Bigram: n -gram of size 2
 - Trigram: n -gram of size 3
- Item:
 - Phonemes
 - Syllables
 - Letters
 - Words
 - Anything else depending on the application.

Example

“the dog smelled like a skunk”

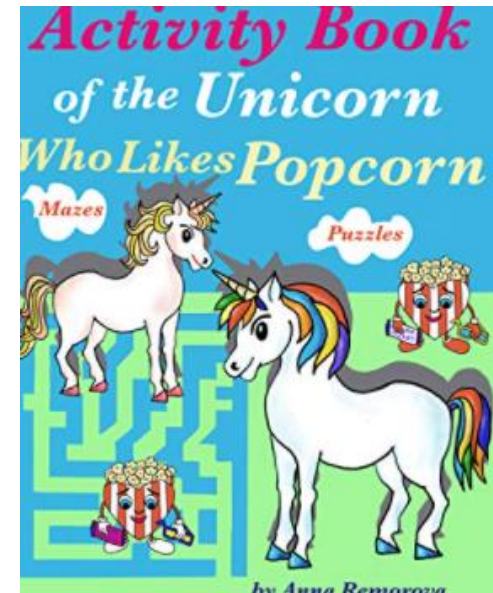
- Bigram:
 - "# the", "the dog", "dog smelled", " smelled like", "like a", "a skunk", "skunk#"
- Trigram:
 - "# the dog", "the dog smelled", "dog smelled like", "smelled like a", "like a skunk" and "a skunk #".

How to predict the next word?

- Assume a language has T word types in its lexicon, how likely is word x to follow word y ?

Solutions:

- Estimate likelihood of x occurring in new text, based on its general frequency of occurrence estimated from a corpus
 - popcorn is more likely to occur than unicorn
- Condition the likelihood of x occurring in the context of previous words (bigrams, trigrams,...)
 - mythical unicorn is more likely than mythical popcorn



Probabilistic Language Modelling

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- A model that computes either of these:

$P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a **language model**.

N-Gram Models

- Models Sequences using the *Statistical Properties* of N-Grams
- An N-gram language model **predicts the probability of a given N-gram within any sequence of words in the language.**
- In other words, using the **previous N-1 words in a sequence we want to predict the next word**

Ex: Sue swallowed the large green _____.

- A. Frog
 - B. Mountain
 - C. Car
 - D. Pill
- **Idea:**
 - given a sequence of letters, what is the *likelihood* of the next letter?
 - From training data, derive a *probability distribution* for the next letter given a *history* of size n.

N Gram – How to compute $P(W)$

- Use the **previous N-1 words in a sequence(history)** to predict the next word
- **So**, Computing $P(w|H)$: *probability of word 'w', given some history 'H'.*

Ex: Consider H is 'its water is so transparent that', and we want to know the probability of next word 'the':

$P(\text{the}|\text{its water is so transparent that})$.

One way to estimate this probability — *relative frequency counts*.

One way to estimate this probability — *relative frequency counts*.

Take a large corpus, count the number of time ‘its water is so transparent that’ and also count the number of times it has been followed by ‘the’.

$$P(\textit{the}|\textit{its water is so transparent that}) = \frac{C(\textit{its water is so transparent that the})}{C(\textit{its water is so transparent that})}$$

The Chain Rule applied to compute joint probability of words in sentence

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$

$P(\text{"its water is so transparent"}) =$

$P(\text{its}) \times P(\text{water} \mid \text{its}) \times P(\text{is} \mid \text{its water})$

$\times P(\text{so} \mid \text{its water is}) \times P(\text{transparent} \mid \text{its water is so})$

How to estimate these probabilities

- Could we just count and divide?

$$P(\text{the | its water is so transparent that}) = \frac{\textit{Count}(\text{its water is so transparent that the})}{\textit{Count}(\text{its water is so transparent that})}$$

- No! Too many possible sentences!
- We'll never see enough data for estimating these

- This method of estimating *probabilities* directly from counts **works fine in many cases** -- However, even the web isn't big enough to give good estimates in most cases.
- Since **language is creative**, and there are new sentences added everyday, which we won't be able to count.
- **Joint probability distribution method to compute probability of entire sequence -- *chain rule of probability***

$$\begin{aligned} P(w_1^n) &= P(w_1)P(w_2|w_1)P(w_3|w_1^2) \dots P(w_n|w_1^{n-1}) \\ &= \prod_{k=1}^n P(w_k|w_1^{k-1}) \end{aligned}$$

- **The chain rule shows the link between computing joint probability of a sequence and computing the conditional probability of word given previous words.**

Intuition of the n-gram model

- The intuition of the n-gram model is that **instead of computing the probability of a word given its entire history, we can approximate the history by just the last few words**
- The **bigram model** approximates the probability of word given all the previous words, by using only the conditional probability of the last preceding word:

$$P(w_n | w_1^{n-1}) \approx P(w_n | w_{n-1})$$

- The assumption that the probability of a word depends only on the previous word is called a *Markov assumption*.

N-gram Model -- a Markov Chain

- Markov models are the class of probabilistic models to predict the probability of some future unit without looking too far into the past.
- We can generalize the bigram (which looks one word into the past) to the trigram (which looks two words into the past) and thus to the n-gram (which looks $n - 1$ words into the past)
- Given the bigram assumption for the probability of an individual word, we can compute the probability of a complete word sequence by substituting

$$P(w_1^n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

- **Markov assumption:** the probability of the next word depends only on the previous k words.

$$P(w_n | w_1 \dots w_{n-1}) = P(w_n | w_{n-k} w_{n-k+1} \dots w_{n-1})$$

($k+1$)-gram or K^{th} order Markov approximation

- Common N-grams:

Unigram: $P(w_1 w_2 \dots w_n) = P(w_1) P(w_2) \dots P(w_n)$

Bigram: $P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_{n-1})$

Trigram: $P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_{n-2} w_{n-1})$

$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$

$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$

Using N-Grams

- For N-gram models
 - $P(w_1, w_2, \dots, w_n)$
 - By the [Chain Rule](#) we can decompose a joint probability, e.g. $P(w_1, w_2, w_3)$ as follows:

$$P(w_1, w_2, \dots, w_n) = P(w_1 | w_2, w_3, \dots, w_n) P(w_2 | w_3, \dots, w_n) \dots P(w_{n-1} | w_n) P(w_n)$$

$$P(w_1^n) \approx \prod_{k=1}^n P(w_k | w_1^{k-1})$$

Estimation of bi-gram probabilities (or n-gram probabilities) -MLE

- An intuitive way to estimate probabilities is called **maximum likelihood estimation** or **MLE**.
- *Get the MLE estimate for the parameters of an n-gram model by getting counts from a corpus, and normalizing the counts so that they lie between 0–1.*
- Ex: To compute a particular bigram probability of a word y given a previous word x , we'll compute the count of the bigram $C(xy)$ and normalize by the sum of all the bigrams that share the same first word x :

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

- Since the sum of all bigram counts that start with a given word w_{n-1} must be equal to the unigram count for that word w_{n-1} :

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1} w_n)}{C(w_{n-1})}$$

Example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Bigram Example 1

- Estimate the likelihood of the sentence:

I want to eat Chinese food.

- $P(\text{I want to eat Chinese food}) = P(\text{I} \mid \langle \text{start} \rangle) P(\text{want} \mid \text{I}) P(\text{to} \mid \text{want}) P(\text{eat} \mid \text{to}) P(\text{Chinese} \mid \text{eat}) P(\text{food} \mid \text{Chinese}) P(\langle \text{end} \rangle \mid \text{food})$
- What do we need to calculate these likelihoods?
 - Bigram probabilities for each word pair sequence in the sentence
 - Calculate from a large corpus

Early Bigram Probabilities from BERP

Eat on	.16	Eat Thai	.03
Eat some	.06	Eat breakfast	.03
Eat lunch	.06	Eat in	.02
Eat dinner	.05	Eat Chinese	.02
Eat at	.04	Eat Mexican	.02
Eat a	.04	Eat tomorrow	.01
Eat Indian	.04	Eat dessert	.007
Eat today	.03	Eat British	.001

The Berkeley Restaurant Project (BeRP) is a testbed for a speech recognition system is being developed by the [International Computer Science Institute](#) in Berkeley, CA

Example 2

- $P(\text{I want to eat British food}) =$
 $P(\text{I} \mid \langle \text{start} \rangle) P(\text{want} \mid \text{I}) P(\text{to} \mid \text{want})$
 $P(\text{eat} \mid \text{to}) P(\text{British} \mid \text{eat}) P(\text{food} \mid \text{British})$
 $= .25 * .32 * .65 * .26 * .001 * .60 = .000080$

- **N-gram models can be trained by counting and normalization**

<start> I	.25	Want some	.04
<start> I'd	.06	Want Thai	.01
<start> Tell	.04	To eat	.26
<start> I'm	.02	To have	.14
I want	.32	To spend	.09
I would	.29	To be	.02
I don't	.08	British food	.60
I have	.04	British restaurant	.15
Want to	.65	British cuisine	.01
Want a	.05	British lunch	.01

N-gram Parameters -Challenges

- Is a high-order n-gram needed?

P(Most biologists and folklore specialists believe that in fact the mythical unicorn horns derived from the narwhal)

- Issues:
 - The *longer* the sequence, the *less likely* we are to find it in a training corpus
 - The larger the n, the more the number of parameters to estimate

Computing Number of Parameters

- Assume that a speaker is staying within a vocabulary of 20,000 words, we want to see the estimates for numbers of parameters

Model	Parameters
1st order (bigram model):	$20,000 \times 19,999 = 400 \text{ million}$
2nd order (trigram model):	$20,000^2 \times 19,999 = 8 \text{ trillion}$
3th order (four-gram model):	$20,000^3 \times 19,999 = 1.6 \times 10^{17}$

Table 6.1 Growth in number of parameters for n -gram models.

- For this reason, *n-gram* systems currently use bigrams or trigrams (and often make do with a smaller vocabulary).